Supervised Learning Report
CS7641: Machine Learning
Spring 2026

1 Assignment Weight

The assignment is worth 12% of the total points.

Read everything below carefully as this assignment has changed term-over-term.

2 Objective

You will build a reproducible supervised learning evaluation harness that produces:

1. An exploratory data analysis (EDA) summary for each dataset,
2. baseline and tuned models for four algorithms,
3. learning and model-complexity curves,
4. runtime/efficiency profiling, and
5. a cross-model comparison.

Your goal is not just to obtain strong metrics, but to justify model behavior using evidence (curves, runtime, bias/variance diagnostics) and to ensure results are leakage-safe and reproducible.

Hypothesis-driven analysis requirement

After completing EDA for each dataset, you must propose at least **one clear hypothesis per dataset** about expected algorithmic performance *before* running full tuning and final experiments. These hypotheses must be grounded in theory from the lectures and readings (e.g., bias–variance, margin-based classifiers, sensitivity to scaling, curse of dimensionality, capacity/regularization, and effects of class imbalance). Your experiments and discussion should be structured around testing these hypotheses, not merely reporting results.

3 Required Datasets for Spring 2026

The following datasets are required for the Spring 2026 cohort and are provided on Canvas. If these datasets are not used, you will receive a zero for the assignment.

Dataset A: Adult Income (Census Income)

- **Task type:** Binary classification
- **Target:** `income` ($\leq 50K$ vs $> 50K$)
- **Notes:** Expect class imbalance ($\sim$ 3:1). The dataset is largely one-hot/binary encoded with a small continuous subset.
- **Required evaluation metrics:** F1 and Accuracy (minimum), plus PR-AUC when reporting threshold-based results (because imbalance makes accuracy insufficient).

Dataset B: Wine Quality (Red + White)

- **Task type:** Multiclass classification
- **Target:** `quality` (discrete rating/class)
- **Notes:** No missing values. Includes a `type` indicator feature (red/white).
- **Required evaluation metrics:** Macro-F1 and Accuracy (minimum). Include a confusion matrix and per-class performance discussion.

Important sanity requirement (both datasets)

If your metrics are *too perfect* (e.g., near-zero error or implausibly high accuracy), you must include a **Leakage / Evaluation Debug** subsection demonstrating that your evaluation is correct. At minimum, include one diagnostic such as:

1. a shuffled-label baseline (performance should collapse),
2. a dummy baseline (most-frequent class), or
3. a train/test swap sanity check.

4 Procedure (What you will do)

You are given two datasets and will complete **exactly two supervised learning tasks** total: **one per dataset**. You will:

1. explore the data and develop hypotheses,
2. build a reproducible preprocessing and evaluation workflow,
3. train and tune four algorithms on each dataset, and
4. write a thorough analysis of your findings.

You need not implement any learning algorithm yourself; however, you must participate in the full process of exploring, tuning, and analyzing.

Hypothesis-first workflow (required)

For each dataset, your workflow should follow this progression:

$$\textbf{EDA} \rightarrow \textbf{Hypothesis} \rightarrow \textbf{Experiments} \rightarrow \textbf{Interpretation}.$$

Your hypotheses should make a concrete prediction (what model family should do well and why), and your analysis should explicitly evaluate whether the results support or contradict your prediction.

What counts as a well-posed hypothesis

A strong hypothesis includes:

1. a predicted ordering or trade-off (e.g., `Linear SVM` $\geq$ `DT` $\geq$ `kNN`),
2. a reason tied to dataset properties observed in EDA (e.g., sparsity, separability, imbalance, noise), and
3. a theory-based explanation (e.g., margin maximization, regularization, bias–variance, locality effects).

Avoid generic claims that cannot be tested.

General rules

- You may program in any language and are allowed to use standard libraries, **as long as they were not written specifically to solve this assignment**. Code repositories, notebooks, or AutoML templates created for CS7641 assignments ("plug-and-play" solutions) are **not allowed**.
- TAs must be able to recreate your experiments on a standard Linux machine if necessary.
- The analysis you provide in the report is paramount and the focus of the assignment.

5 Required Algorithms (run all required components on both datasets)

You must evaluate the following learning algorithms on each dataset:

- **Decision Trees (DT).** Use some form of pruning/regularization (e.g., post-pruning via ccp_α, depth limits, leaf constraints). You are not required to use information gain. Gini or entropy are fine. **State what you use and why**.
- **k-Nearest Neighbors (kNN).** Compare **meaningfully different** values of k (e.g., small/medium/large) and justify your choice.
- **Support Vector Machines (SVM).** Evaluate ≥ 2 **kernels** per dataset (e.g., linear vs. RBF) and tune C and γ (where relevant).
- **Neural Networks (MLP; SGD only).** You must train **two** neural network implementations per dataset:
  - **One scikit-learn model:** `MLPClassifier` using **SGD only**.
  - **One PyTorch model:** a compact MLP trained with **SGD only**.

  You will compare capacity scaling (depth vs. width) while keeping SGD constant, and you must hold the total parameter count approximately constant when comparing architectures. **No momentum, no Nesterov, and no adaptive variants such as Adam/Adagrad/RMSprop are allowed.** Report learning rate, batch size, epochs/early stopping, and regularization choices.

6 Required Workflow Outputs (per dataset, per algorithm)

Applies to both datasets, for each required algorithm component (DT, kNN, SVM, and NN-Sklearn & NN-PyTorch).

Data, targets, and leakage controls

- Declare the target, the task type, and justify your chosen metrics.
- Report the class distribution and describe how imbalance affects your evaluation.
- State leakage controls you applied (what you removed, what you verified, and why).
- Use a single held-out test split once at the end. Perform tuning using cross-validation (or a train/validation split) on the training data only.
- State at least one dataset-specific hypothesis *before* presenting tuned results, and explicitly revisit it in your conclusions.

Methodology & reproducibility

- Fix random seeds, record exact splits, and make preprocessing deterministic.
- Provide sufficient instructions to reproduce your results on a standard Linux machine.

Per-algorithm required figures/tables

- **Learning curves (LC):** training and validation metric vs. training size. Diagnose bias/variance.
- **Model-complexity curves (MC):** validation metric vs. one hyperparameter (e.g., NN width/L2; SVM C/γ; kNN k; DT depth/ccp_α).
- **Runtime table:** fit and predict wall-clock times with a brief hardware note.
- **Classification add-ons:** confusion matrix at a justified operating point; per-class discussion for multi-class results.

Algorithm-specific essentials

- **NN (both implementations):** show epoch-based learning curves (train vs. validation loss/metric) and discuss early stopping and regularization.
- **NN comparison requirement:** explicitly compare scikit-learn vs. PyTorch results and discuss differences in optimization behavior, stability, runtime, and tunability under **SGD-only constraints**.
- **SVM:** scale features; use ≥ 2 kernels per dataset; tune C and kernel parameters.
- **kNN:** scale features; justify distance metric and weighting; discuss sensitivity to irrelevant features and high dimensionality.
- **DT:** show a pruning/regularization curve and report final depth and number of leaves.

Cross-model comparison (discussion required)

Compare across algorithms and datasets with evidence-backed trade-offs: metric vs. wall-clock, stability vs. capacity, sensitivity to hyperparameters, and effect of preprocessing. Tie conclusions to course concepts.

Conclusion (required)

Discuss:

- Type I/II errors in the context of your operating point (classification),
- what you learned about model behavior on each dataset, and
- one realistic next step.

7 Extra Credit (up to 5 points; NN only; *SGD only*)

- **Activation function study (one dataset only).** On exactly *one* dataset of your choice (Adult *or* Wine), evaluate **four** activation functions (e.g., ReLU, GELU, SiLU/Swish, tanh) using the *same* architecture, preprocessing, regularization, and training protocol. Use **SGD only**. *Deliverables:* epoch-wise validation curves for each activation, a comparison table of final metrics, and a brief analysis of convergence speed/stability and trade-offs under SGD.

8 External Sources & Literature (required)

- Include outside sources beyond course materials, with at least **two peer-reviewed references**.
- Use these sources to **justify choices** (metrics for imbalance, threshold selection, pruning, kernel decisions, leakage controls) and to **interpret results**.
- Acceptable citation styles: **MLA, APA, or IEEE**. Pick one and be consistent.

9 Report Requirements

Your report is a formal technical paper limited to **8 pages total**, including figures and references. Anything past 8 pages will not be read.

Your report must be written in LaTeX on Overleaf. When submitting your report, you are required to include a **READ-ONLY** link to the Overleaf Project. If a link is not provided in the report or Canvas submission comment, 5 points will be deducted from your score. Do not share the project directly via email.

Interpret, don't summarize

Every figure/table must include a takeaway tied to algorithm behavior and the data. Avoid repeating the legend; explain what the result *means* and why it happened.

Formal writing requirement

Your *report* must read like a technical paper, not a checklist. Bullets are allowed when they improve clarity (e.g., listing hyperparameters, summarizing preprocessing steps, or describing an experimental protocol), but your **analysis and conclusions must be written in paragraphs** with coherent reasoning. Excessive bullet-only writing that replaces interpretation (especially in Results/Discussion) will be treated as a draft and scored accordingly. As a rule of thumb: if a section could be pasted into a spreadsheet and still make sense, it is not sufficient prose. If there are any use of excess bullets, we will deduct 50 points from the report total.

10 Acceptable Libraries

You may use any standard open-source libraries that support reproducible experimentation and were not written specifically to solve this assignment. The list below includes common examples.

Recommended stack (examples)

- **Core ML:** `scikit-learn` (pipelines, cross-validation, metrics, calibration), `imbalanced-learn` (imbalance utilities), `PyTorch` (required).
- **Data & plotting:** `pandas`, `polars`, `NumPy`, and `matplotlib` (or `plotly`/`altair`/`seaborn`).

Neural network optimizer rule (required)

For **all neural network experiments**, you must use **SGD only**. Momentum, Nesterov acceleration, and adaptive optimizers (Adam/Adagrad/RMSprop/etc.) are not allowed.

11 Submission Details

All scored assignments are due by the time and date indicated on Canvas (Eastern Time, ET). Canvas displays times in your local time zone if your profile is set correctly; grading deadlines are enforced in ET.

All assignments are due at 11:59:00 PM ET on the final Sunday of the unit. Submissions received by **7:59:00 AM ET** the next morning are accepted **without penalty**.

Late penalty

After the grace window, the score is reduced **20 points per calendar day**. Treat **11:59 PM ET** as your real deadline; allow time for upload and verification.

What to submit

You will submit **two PDFs**:

1. **SL_Report_{GTusername}.pdf** — your report (Overleaf).
2. **REPRO_SL_{GTusername}.pdf** — your reproducibility sheet, which must include:
   (a) A **READ-ONLY** link to your Overleaf project.
   (b) A **GitHub commit hash** (single SHA) from the final push of your code.
   (c) **Exact run instructions** to reproduce results on a standard Linux machine (environment setup, commands, data paths, and random seeds).
   (d) **EDA summaries** for both datasets (Adult and Wine), included directly in this PDF.

- Include the READ-ONLY Overleaf link in the report or Canvas submission comment. **Do not send email invitations.**
- Use the **GT Enterprise GitHub** for course-related code.
- Provide sufficient instructions to retrieve code and data (Canvas paths and file names are sufficient).

12 Feedback Requests

When your assignment is scored, you will receive feedback explaining errors and successes. If you are confused by feedback, start a private thread on Ed.

13 Plagiarism and Proper Citation

The easiest way to fail this class is to plagiarize. **Using the analysis, code, or graphs of others in this class is considered plagiarism**. We care about your analysis: it must be original and grounded in your own experiments.

If you copy any amount of text from other students, websites, or any other source without proper attribution, that is plagiarism. Citing is required but does not permit copying large blocks of text. All citations must use a consistent style (IEEE/MLA/APA).

LLMs (disclosure required)

We treat AI based assistance the same way we treat collaboration with people. You may discuss ideas and seek help from classmates, colleagues, and AI tools, but all submitted work must be your own. For this course a large language model is any model with more than one billion parameters. These tools can increase productivity, but they are aids, not substitutes for your skills. The goal of reports is synthesis of analysis, not merely getting an algorithm to run. Even if you locate code elsewhere or generate code with an AI tool, you must apply it thoughtfully to your data and write the analysis yourself.

Every submission must include an AI Use Statement. List the tools used and what they assisted with and confirm that you reviewed and understood all assisted content. If an editor or platform surfaces suggestions, you may keep them only with disclosure and verification. You are not required to disable assistants. Be aware that coding editors often include assistants by default. Google Colab and Visual Studio Code commonly surface them. Also check any writing or grammar platforms you use. Overleaf through Georgia Tech is acceptable.

Allowed with disclosure. Brainstorming, outlining, grammar and clarity edits, code generation, code refactoring, and debugging.

Not allowed. Submitting AI written analysis, conclusions, or figures as your own. Fabricating results or citations. Paraphrasing AI or prior work to evade checks. Uploading course materials or private data to public tools.

Example Statement at the very end of the report before References. "AI Use Statement. I used ChatGPT and Visual Studio Code Copilot to brainstorm and outline sections of the report, generate and refactor small code snippets, debug a pandas indexing issue, and edit grammar and clarity throughout. I reviewed, verified, and understand all assisted content."

Verification will rely on provenance and reproducibility. We will not ask for live coding. We will review platform histories, including Overleaf revision timelines and copy and paste patterns, and Git commit and branch history. We may request your repository with full history, Overleaf project history, notebooks, logs, and intermediate outputs, and we may run reproducibility checks. Provide run instructions, environment details, random seeds, and the scripts used to generate figures and tables. Similarity and AI indicators, including Turnitin, are corroborating signals only and are not used in isolation.

If we suspect a violation, we will document evidence, notify you, and request artifacts. Cases may be referred to the Office of Student Integrity (OSI). Undisclosed or prohibited use is academic misconduct. This policy may be updated during the term. When in doubt, always use proper citations, and ask clarifying questions on Ed.

14 Version Control

- v1.0 – 01/16/2026 – TJL finalized SL Report for Spring 2026 term.

Assignment description written by Theodore LaGrow.